Adaptive Advance-Time Control for nFAPI Split under Unpredictable Latency

Ming-Hong HSU*, Ray-Guang Cheng*, and Co-author 1†
*Dept. of Electronic and Computer Engineering, National Taiwan University of Science and Technology, Taiwan
†Affiliation 2
Email: crg@mail.ntust.edu.tw

Abstract—
Index Terms—nFAPI, O-RAN, Functional Split, Adaptive Control.

I. Introduction

A. Background and Motivation

To address the diverse demands of 5G networks, the 3rd Generation Partnership Project (3GPP) and the O-RAN Alliance have standardized the disaggregation of traditional base stations into Open Central Units (O-CUs), Open Distributed Units (O-DUs), and Open Radio Units (O-RUs) [1]. Among the proposed architectures, three functional split configurations have emerged as mainstream: Option 2 (O-CU/O-DU), Option 7.2 (O-DU/O-RU), and Option 6 (O-DU High/Low) [2].

These splits present distinct synchronization requirements and management strategies. Option 2 relies on sequence numbers and deep buffering over general packet-switched networks, making its timing constraints relatively relaxed. Option 7.2 [3], operating within the physical layer, demands strict phase/symbol-level synchronization enforced by hardware Precision Time Protocol (PTP) over Time-Sensitive Networking (TSN). Occupying a unique intermediate position is Option 6 (standardized as nFAPI by the Small Cell Forum [4]), which separates the MAC and PHY layers. Option 6 requires frame/slot-level synchronization and focuses on delay management, offering the flexibility to operate over general packet-switched networks without the prohibitive costs of TSN.

The rationale for deploying the MAC layer (O-DU High) in a cloud environment via Option 6 is primarily driven by resource pooling, as Layer 2 processing complexity scales dynamically with user equipment (UE) density [5]. However, migrating the MAC layer to a Virtualized Network Function (VNF) introduces endogenous processing latency that compresses the available margin for fronthaul transmission delay. Unlike Option 7.2, which rigidly drops out-of-window packets, Option 6 must adaptively manage delays to ensure slot-level synchronization over non-deterministic Ethernet transport. Existing open-source implementations typically employ static timing advance configurations, which fail to accommodate dynamic network jitter and server processing variations, leading to frequent scheduling failures (e.g., missing subframes).

This highlights a fundamental gap in current disaggregated RAN designs: the lack of adaptive timing control mechanisms for intermediate-layer splits operating under non-ideal transport environments.

B. Related Works

The evolution of Radio Access Network (RAN) architectures requires a thorough study of functional splits [2]. A fundamental challenge in deploying disaggregated RAN is managing fronthaul latency constraints [6]. Recent work on cell-free massive MIMO in O-RAN analyzes fronthaul planning under functional split 7.2x, highlighting the trade-off between capacity and deployment cost [7]. However, in these 7.2x-based architectures, stringent timing requirements dictate that packets arriving outside the Radio Unit (RU) timing window must be passively discarded (i.e., out-of-window packet discard), making them highly vulnerable to network jitter. In contrast, Option 6 (nFAPI) offers the flexibility to operate over general packet-switched networks. By enabling microsecond-level, active phase and timing adjustments at the MAC–PHY boundary, Option 6 allows the system to proactively absorb fronthaul jitter rather than passively dropping late packets [8].

Recent studies address latency from various architectural levels. Adamuz-Hinojosa et al. [9] proposed an orchestration framework for 6G O-RAN to bound queuing delays, operating on a higher orchestration timescale. Virtualizing the MAC scheduler introduces significant server processing variations, which Lv et al. [10] addressed by jointly optimizing scheduling and orchestration to guarantee QoS. However, their optimization predominantly targets computational resource contention rather than dynamically adapting to transport network jitter.

The severe constraints of sub-millisecond scheduling in disaggregated architectures are demonstrated by Chen et al. [11]. Their work illustrates that static timing configurations are highly susceptible to missing subframe errors under network jitter. This highlights the critical gap filled by our research. While existing literature focuses on high-level orchestration, our work specifically addresses the microsecond phase-level synchronization challenges inherent to Option 6. We propose a dynamic Adaptive Advance-Time Control algorithm and validate it on an

end-to-end commercial testbed, proving its efficacy against non-deterministic fronthaul jitter and system variations.

C. Research Scope and Contributions

In this paper, we focus on the nFAPI-based Option 6 split as a representative case of timing-sensitive yet transport-flexible RAN disaggregation. While Option 6 offers significant advantages in terms of reduced fronthaul bandwidth and deployment flexibility, its performance is highly sensitive to non-deterministic latency introduced by transport networks and server processing.

We identify that the root cause of performance degradation in such systems lies in the inability of the MAC scheduler to adapt its transmission timing to dynamic end-to-end latency variations. Existing open-source implementations, such as OpenAirInterface (OAI), typically employ static timing advance configurations, which fail to accommodate fluctuating network conditions, leading to frequent synchronization failures and throughput degradation.

To address this issue, we propose an adaptive timing control framework that dynamically adjusts scheduling lead time based on real-time latency estimation. The proposed approach introduces a dual-loop architecture, where a convergence optimization loop estimates end-to-end delay characteristics, and a timing actuator dynamically compensates for timing offsets to ensure that scheduling messages arrive within the valid timing window.

The main contributions of this paper are summarized as follows:

- Systematic Analysis of Timing Uncertainty in Split 6: We characterize the impact of non-deterministic latency, including network jitter and processing delay, on synchronization stability in disaggregated RAN systems.
- Adaptive Advance-Time Control Algorithm: We propose a dynamic timing adjustment mechanism that continuously aligns scheduling decisions with varying transport delays, effectively mitigating timing violations.
- Experimental Validation on OAI Platform: We implement the proposed framework in a real-world testbed and demonstrate that it eliminates missing subframe errors and significantly improves throughput stability under non-ideal fronthaul conditions.

II. System Model

Before diving into the system details, this study introduces the adopted system framework. To integrate the Network Functional Application Platform Interface (nFAPI) with mainstream commercial equipment, the proposed system adopts a resilient two-stage split architecture. The network first connects via the Split 6 nFAPI interface between the network functions, and then translates to the standard O-RAN Split 7.2x interface to connect to a commercial Pegatron O-RU. Figure 1

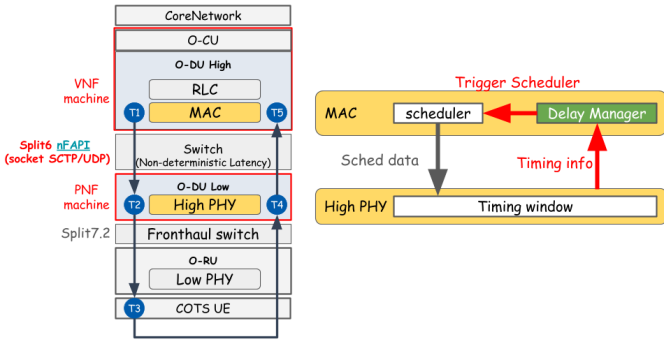


Fig. 1. System architecture and delay management mechanism.

illustrates this proposed system architecture and its delay management mechanism.

A. End-to-End Architecture

The system establishes a complete 5G end-to-end communication link, extending from the core to the user equipment:

- Core Network (CN): The central element responsible for routing, session management, and authenticating user data.
- O-RAN Central Unit (O-CU): A logical node that handles higher-layer protocol stacks, such as the Radio Resource Control (RRC) and Packet Data Convergence Protocol (PDCP). It connects with distributed units via the F1 interface.
- O-RAN Distributed Unit (O-DU): To optimize computational performance and deployment flexibility, the system separates the O-DU into an O-DU High and an O-DU Low. The O-DU High is executed as a Virtualized Network Function (VNF) on a virtualized machine, whereas the O-DU Low is deployed as a Physical Network Function (PNF) on dedicated hardware.
- Switch: A commercial network switch is located between the VNF and PNF. This mid-layer connection utilizes the nFAPI protocol. However, traversing this packet-switched network inevitably generates non-deterministic latency.
- O-RAN Radio Unit (O-RU): A specialized hardware unit responsible for low-level physical layer (Low PHY) processing, digital beamforming, and Radio Frequency (RF) transmission.
- Commercial Off-The-Shelf User Equipment (COTS UE): Standard commercial smartphones or modems that act as the terminal receivers in the network.

B. O-DU High: MAC and Delay Manager

Within the Media Access Control (MAC) layer of the O-DU High, the core components handling timing mechanisms include the Scheduler and the nFAPI Delay Manager:

- Scheduler: Critical for allocating the available radio resources (such as time-frequency resource blocks) and determining the exact priority and timing of downlink/uplink data transmissions.
- nFAPI Delay Manager: This resides as the crucial control unit of this delay-aware architecture. Because unpredictable, unstable latency exists in the switch between the VNF and PNF, the Delay Manager continuously receives timing feedback from the lower physical layer. It uses this to dynamically calculate network jitter and proactively adjust the slot-ahead (S_{ahead}) to trigger the scheduler. This calculation guarantees that the scheduling instructions (Sched data) are generated ahead of time to account for network delays.

C. O-DU Low: High PHY and Timing Window

The O-DU Low manages the higher-level baseband processing of the physical layer (High PHY). At this level, the most critical synchronization mechanism is the nFAPI PNF Timing Window:

- Timing Window: A predefined, strict time duration. When the scheduling message (specifically the DL_TTI.request) sent by the O-DU High propagates across the network, it must accurately arrive and fall exactly within this Window. Otherwise, the High PHY will reject the message and fail to process the subframe data.
- Timing Info (Δt_{arrive}): The PNF continuously monitors the exact arrival offset of control messages relative to the transmission deadline. This offset, denoted as Δt_{arrive}, represents the remaining time margin within the timing window. The PNF periodically sends this information back to the Delay Manager in the MAC layer. This establishes a closed-loop feedback control system, compensating for the unpredictable transmission latency variations between the VNF and PNF machines.

D. Key Validation Metrics

To evaluate the stability, synchronization accuracy, and real-time performance of the proposed algorithm under non-ideal environments, this study defines the following Key Validation Metrics:

- MAC Layer HARQ RTT ($T_5 - T_1$): Measures the Hybrid Automatic Repeat Request (HARQ) loop delay specifically at the MAC layer. This metric accurately reflects the feedback efficiency and operational health of the link layer.
- VNF-PNF DL Latency ($T_2 - T_1$): Measures the one-way downlink latency accumulated from the virtualized machine (VNF) to the physical machine (PNF). This metric is used to directly quantify and analyze the delay jitter impact injected by the intermediate Ethernet switch.

Note that RTT denotes the Round-Trip Time. To validate the split-architecture stability under non-ideal fronthaul conditions, network latency simulation is conducted at the uplink ($T_5 - T_4$) and downlink ($T_2 - T_1$) segments via Linux Traffic Control (TC).

To validate the proposed performance optimization within a realistic network environment, the system design leverages a commercial-grade deployment framework:

- O-RAN Split 7.2x Interoperability: While the Small Cell Forum (SCF) has standardized the nFAPI for the MAC-PHY split (Option 6) [8], commercial-grade Open Radio Units (O-RUs) natively support the O-RAN Split Option 7.2x. To achieve seamless integration with open-source stacks like OpenAirInterface [12], our system bridges these interfaces.
- Hybrid System Architecture for E2E Validation: To validate the nFAPI performance over a fully commercial ecosystem, this paper adopts a hybrid architecture (Split Option 6 + Split Option 7.2). This setup establishes a translation/interworking layer that enables the nFAPI-driven MAC/High-PHY to successfully interact with a commercial Pegatron O-RU, facilitating end-to-end verification using Commercial Off-The-Shelf (COTS) UEs.

III. Proposed Method

This section details the proposed dynamic timing adjustment algorithm. To resolve the unstable latency during packet transmission between the PNF and VNF, we propose a Dual-Loop Timing Control Architecture.

A. Dual-Loop Architecture

The timing control of this system adopts a "policy-execution separation" dual-loop architecture, ensuring that the timing advancement on the VNF side can flexibly cope with network jitter without inducing unexpected oscillations:

1) Convergence Optimization Loop (Policy): Dynamically analyzes the statistical data reported by the PNF through the convergence optimization algorithm to determine the target slot-ahead S_{next}.
2) Timing Actuator Loop (Execution): Executed by an independent VNF timing thread. This thread is the only unit in the system capable of altering the absolute sleep time and slot propagation, translating the upper-layer policy target (S_{next}) and low-layer microsecond phase deviations into actual timing advancement.

B. VNF Timing Actuator

The core function of the VNF Timing Actuator is isolation and precise phase execution. Initially, we define the basic parameters. Let μ denote the Numerology, and the slot duration (in μs) is defined as:

$$\text{slot\_duration} = \frac{1000}{2^{\mu}} \tag{1}$$

The target slot-ahead value, S_{next}, is determined by the policy loop. For microsecond-level phase correction, the actuator calculates the next absolute wake-up time T_{next}:

$$T_{\text{next}}^{(k)} = T_{\text{next}}^{(k-1)} + \text{slot\_duration} + \Delta_{\text{pending}} \quad (2)$$

where Δ_{pending} is the sub-slot phase deviation. By adjusting the absolute wake-up time, it translates S_{next} and phase deviations into actual timing advancement.

C. Adaptive Node Synchronization

To achieve microsecond-level phase alignment, the system utilizes an IEEE 1588 (PTP) style timestamp exchange. When the VNF receives the UL_node_sync message, it calculates the instantaneous timing offset ($offset$) based on four timestamps (t_1 to t_4). To avoid transient spikes, the $offset$ and its variation ($error$) are smoothed using an Exponentially Weighted Moving Average (EWMA) model to produce the mean offset (μ_{offset}) and the mean absolute deviation (dev_{offset}), utilizing smoothing factors α and β. A dynamic synchronization threshold ($Threshold$) is then derived directly from the estimated jitter (dev_{offset}). If the absolute $offset$ exceeds this $Threshold$, the system breaks the $offset$ down into a slot adjustment (S_{adj}) and a microsecond-level adjustment (μs_{adj}) based on the slot duration (slot_duration). These parameters are subsequently consumed by the VNF Timing Actuator to precisely align the local clock. The process is summarized in Algorithm 1.

Algorithm 1 Adaptive Node Synchronization

Require: t_1, t_2, t_3 (Timestamps from PNF UL_node_sync), t_4 (VNF local receive timestamp)
Ensure: S_{adj} (Slot adjustment), μs_{adj} (Microsecond adjustment)
Notation:
$offset$ (Instantaneous timing offset), μ_{offset} (EWMA mean offset)
dev_{offset} (EWMA mean absolute deviation / Jitter)
$Threshold$ (Dynamic synchronization threshold)
α, β (Smoothing factors, where $\alpha = 1/8, \beta = 1/4$)
slot_duration (Slot duration in μs)
Phase I: Timestamp Processing and Wrap-around Protection
$d_1 \leftarrow t_2 - t_1$
$d_2 \leftarrow t_4 - t_3$
Phase II: Offset and Dynamic Jitter Estimation
$offset \leftarrow (d_1 - d_2)/2$
$error \leftarrow offset - \mu_{offset}$
$\mu_{offset} \leftarrow \mu_{offset} + \alpha \cdot error$
$dev_{offset} \leftarrow dev_{offset} + \beta(|error| - dev_{offset})$
$Threshold \leftarrow dev_{offset}$
Phase III: Phase Alignment
if $|offset| > Threshold$ then
    $S_{adj} \leftarrow \lfloor offset/\text{slot\_duration} \rfloor$
    $\mu s_{adj} \leftarrow offset \pmod{\text{slot\_duration}}$
    Apply S_{adj} and μs_{adj} to adjust local clock
else
    State $\leftarrow$ Synchronized
    $S_{adj} \leftarrow 0$, $\mu s_{adj} \leftarrow 0$
end if
return $S_{adj}, \mu s_{adj}$

D. Trigger Timing Control (Convergence Optimization)

To counteract fluctuating network latency and maintain scheduling robustness, this paper proposes a dynamic trigger timing control policy. Upon receiving timing statistics, specifically the worst-case late arrival offset (Δt_{arrive}) reported by the PNF, the system computes the deviation ($error$) and incorporates an EWMA model inspired by the Jacobson/Karels TCP RTT algorithm to estimate the mean latency (μ_{delay}) and jitter deviation (dev_{delay}), utilizing smoothing factors α and β.

Before making tracking adjustments, the algorithm establishes multiple Panic Criteria. A statistical anomaly is detected if the absolute $error$ exceeds $3 \cdot dev_{delay}$, indicating a sudden latency spike. A strict violation occurs if $\Delta t_{arrive} >$ slot_duration, meaning the packet is late beyond the slot boundary. Either condition triggers a Fast Attack Panic Mode, radically incrementing the current trigger timing (S_{curr}) by 1 slot to calculate the next trigger timing (S_{next}) to ensure connection continuity and mitigate catastrophic deadline violations.

Conversely, to maximize low-latency operation in safe zones, a Proactive Latency Reduction mechanism is used. When the latency operates safely below an absolute boundary ($\Delta t_{arrive} \leq$ Safe_Margin), a stability counter ($count_{stable}$) increments. Once this counter exceeds an adaptive stability threshold (N_{stable}) which scales dynamically by dev_{delay}, the system cautiously drops S_{curr} by 1 to determine S_{next}. This asymmetric response (fast expansion, slow decay) prevents ping-pong oscillation while minimizing round-trip time. Finally, S_{next} is limited within the allowable nFAPI timing window. The adjustment strategy is condensed in Algorithm 2.

IV. Experimental Results

Experiments were conducted to verify the proposed delay management method in an OpenAirInterface (OAI) nFAPI End-to-End environment. Each reported throughput value represents the average of 180 data points (3 independent trials × 60 samples per trial). We structure the evaluation into three parts: validating the EWMA mechanism (Section IV-B), benchmarking peak throughput and MAC Round-Trip Time (RTT) against static baselines (Section IV-C), evaluating stability under network congestion (Section IV-D), and confirming the end-to-end functional deployment through an rApp demonstration video.

Algorithm 2 Trigger Timing Control

Require: Δt_{arrive} (Timing offset info from PNF)
Ensure: S_{next} (Trigger timing for the next MAC scheduling)
Notation:
S_{curr} (Current trigger timing), μ_{delay} (EWMA mean delay)
dev_{delay} (EWMA mean deviation)
$count_{stable}$ (Consecutive stable arrivals counter)
N_{stable} (Adaptive stability threshold, dynamically scaled by dev_{delay})
α, β (Smoothing factors, where $\alpha = 1/8, \beta = 1/4$)
Safe_Margin (Target safety margin), slot_duration (Slot duration)
Phase I: Delay Statistics Update
$error \leftarrow \Delta t_{arrive} - \mu_{delay}$
$\mu_{delay} \leftarrow \mu_{delay} + \alpha \cdot error$
$dev_{delay} \leftarrow dev_{delay} + \beta(|error| - dev_{delay})$
Phase II: Anomaly Detection
$IsAnomaly \leftarrow (|error| > 3 \cdot dev_{delay}) \vee (\Delta t_{arrive} >$ slot_duration)
Phase III: Timing Adjustment
if $IsAnomaly$ then
  $S_{next} \leftarrow S_{curr} + 1$
  $count_{stable} \leftarrow 0$
else if $\Delta t_{arrive} \leq$ Safe_Margin then
  $count_{stable} \leftarrow count_{stable} + 1$
  if $count_{stable} \geq N_{stable}$ then
    $S_{next} \leftarrow S_{curr} - 1$
    $count_{stable} \leftarrow 0$
  else
    $S_{next} \leftarrow S_{curr}$
  end if
else
  $S_{next} \leftarrow S_{curr}$
  $count_{stable} \leftarrow 0$
end if
Limit S_{next} within the allowable nFAPI timing window
return S_{next}

TABLE I
Testbed Node Configuration

Unit	Specification
Core Network (CN)	Open5GS
SW Stack / Branch	OpenAirInterface (2026.w13)
VNF Server	Intel® Xeon® Gold 6226R
PNF Server	Intel® Xeon® Gold 6433N
O-RU	Pegatron RU (P5G-Indoor O-RU-PR1450)
COTS UE	Samsung Galaxy S20

TABLE II
Wireless System Parameters

Parameter	Setting
Subcarrier Spacing	30 kHz
TDD Pattern	3D1S1U
MIMO / Ports	4-layer / 4T4R
Fronthaul	O-RAN F release

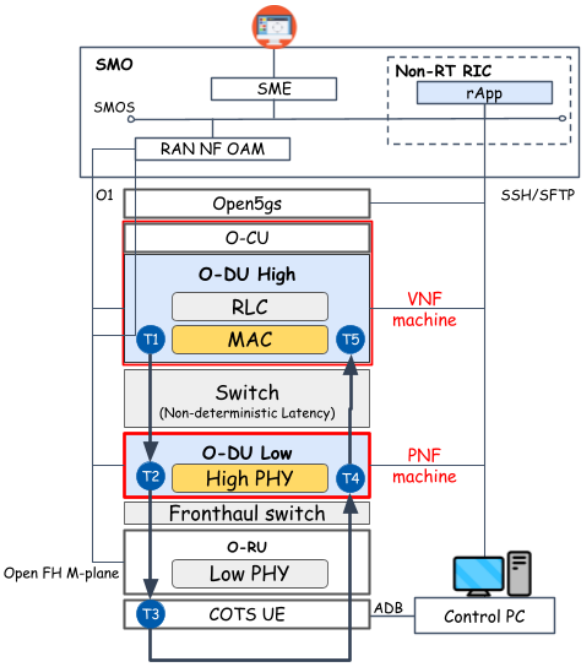


Fig. 2. Architecture of the rApp automated evaluation framework.

A. Experimental Scenario and rApp Framework

The testbed environment connects a series of high-performance servers, radio units, and a commercial smartphone. The detailed software and hardware configurations for the end-to-end network deployment are outlined below.

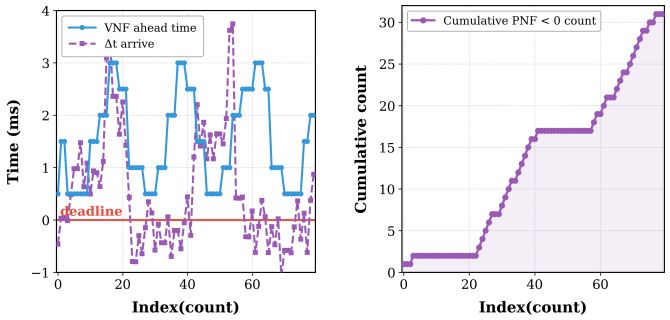
To ensure repeatable validation, an automated evaluation framework (rApp) was developed, as shown in Fig. 2. This rApp automates end-to-end experiments, data collection, and chart generation. Crucially, it manages the network latency simulation at $T_5 - T_4$ and $T_2 - T_1$ via Linux Traffic Control (TC) to systematically validate the split-architecture stability.

B. Scenario I: Validation of EWMA for PNF Timing Info

This scenario validates the application of an Exponentially Weighted Moving Average (EWMA) to smooth the PNF Timing Info. Without EWMA processing, the raw timing information exhibits high variance due to network jitter, as shown in Fig. 3(a). Applying EWMA effectively filters out this high-frequency jitter (Fig. 3(b)), providing a stable baseline for the dynamic timing adjustment algorithm.

C. Scenario II: Performance Benchmark

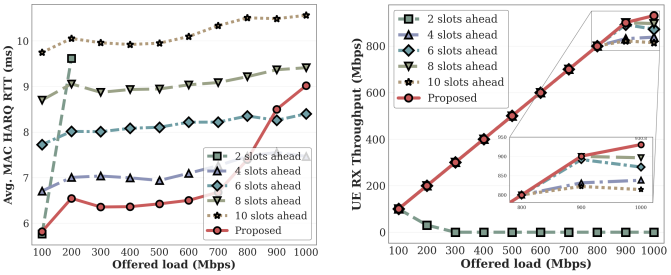
We benchmark the peak throughput and MAC HARQ RTT against a static baseline. Under stable fronthaul



(a) Non-applied EWMA (b) Applied EWMA

Fig. 3. Validation of EWMA processing on PNF Timing Info.

conditions, as shown in Fig. 4(a), the adaptive method maintains better RTT efficiency across different offered loads with minimal packet loss. Furthermore, Fig. 4(b) demonstrates that the adaptive mechanism achieves the best throughput performance at peak loading by mitigating transient jitter from the server operating system.



(a) MAC Latency & Reliability (b) Throughput Optimization

Fig. 4. Performance evaluation: MAC RTT efficiency and system throughput.

D. Scenario III: Stability under Network Congestion

Under simulated network congestion, the adaptive control dynamically adjusts the trigger timing to compensate for increased one-way delay. Fig. 5 illustrates that while static methods fail when jitter exceeds the timing window, our approach maintains synchronization and stable throughput by proactively advancing the scheduling cycle.

E. Functional Deployment Demonstration

To confirm the functional deployment of the proposed system, a demonstration video showcases the automated rApp lifecycle, including real-time latency monitoring and dynamic timing adjustment. This verification confirms that the adaptive algorithm successfully maintains link stability in a virtualized environment.

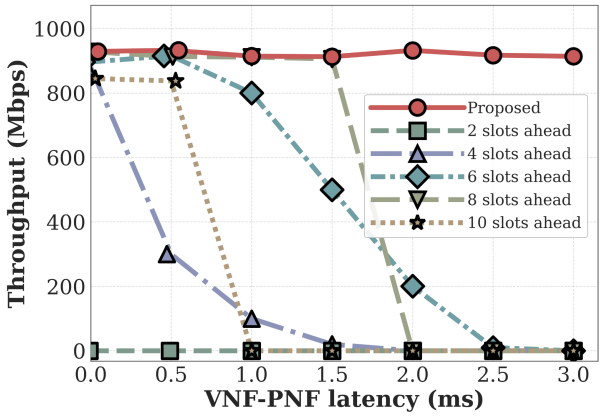


Fig. 5. System stability and throughput performance under varying network congestion.

References

[1] 3GPP, "Study on new radio access technology: Radio access architecture and interfaces," 3rd Generation Partnership Project (3GPP), Tech. Rep. TR 38.801, 2017, version 14.0.0.

[2] K. Alam, M. A. Habibi, M. Tammen, D. Krummacker, W. Saad, M. D. Renzo, T. Melodia, X. Costa-Pérez, M. Debbah, A. Dutta, and H. D. Schotten, "A comprehensive tutorial and survey of o-ran: Exploring slicing-aware architecture, deployment options, use cases, and challenges," IEEE Communications Surveys & Tutorials, vol. 28, pp. 1637–1678, 2026.

[3] O-RAN Alliance, "O-ran fronthaul working group control, user and synchronization plane specification," O-RAN Alliance, Tech. Rep. O-RAN.WG4.CUS.0-v11.00, 2023.

[4] S. C. Forum, "5g network fapi (nfapi) specification," Small Cell Forum, Tech. Rep. SCF225, 2020.

[5] X. Foukas et al., "Computational resource consumption of 5g virtualized ran," in 2021 IEEE International Conference on Communications (ICC). IEEE, 2021.

[6] L. M. P. Larsen, A. Checko, and H. L. Christiansen, "A survey of the functional splits in 5g next-generation radio access networks," IEEE Communications Surveys & Tutorials, vol. 21, no. 3, pp. 2349–2372, 2019.

[7] A. S. Mohammed, K. S. Tharakan, H. A. Ammar, H. ElSawy, and H. S. Hassanein, "Fronthaul network planning for hierarchical and radio-stripes-enabled cf-mmimo in o-ran," IEEE Transactions on Wireless Communications, vol. 25, pp. 14 781–14 796, 2026.

[8] V. G. Douros, A. Garcia-Saavedra, and C.-P. Xavier, "nfapi: The standard interface for sdn-based 5g small cell networks," IEEE Communications Standards Magazine, vol. 2, no. 3, pp. 32–40, 2018.

[9] O. Adamuz-Hinojosa, L. Zanzi, V. Sciancalepore, A. Garcia-Saavedra, and X. Costa-Pérez, "Oranus: Latency-tailored orchestration via stochastic network calculus in 6g o-ran," in IEEE INFOCOM 2024 - IEEE Conference on Computer Communications, 2024, pp. 61–70.

[10] J. Lv, Y. Gao, Z. Ding, Y. Lin, X. You, G. Yang, and W. Dong, "Providing ue-level qos support by joint scheduling and orchestration for 5g vran," in IEEE INFOCOM 2024 - IEEE Conference on Computer Communications, 2024, pp. 51–60.

[11] Y. Chen, Y. T. Hou, W. Lou, J. H. Reed, and S. Kompella, "M3: A sub-millisecond scheduler for multi-cell mimo networks under c-ran architecture," in IEEE INFOCOM 2022 - IEEE Conference on Computer Communications, 2022.

[12] X. Wang et al., "An open-source implementation of the 5g nr functional split option 6," in 2022 IEEE International Conference on Communications (ICC), 2022, pp. 1–6.